Lab 7: Fuzzing for Input Validation Bugs

Kaushik Krishna Dwarakanath | PES1UG24CS900 | Section 5-B
Software Engineering | Phone Repair Quote | 11 November 2026

Objective: Use Hypothesis to identify unexpected-input failures in the three supplied parsers, harden them, and verify both crash resistance and expected outputs. File names follow the assignment repository README.

Environment: Python 3.15.0, pytest 9.2.0, Hypothesis 6.170.15, Linux. Report generated with ReportLab 5.1.0. Dependencies are pinned in requirements.txt; tests run in a local virtual environment.

Function	Observed failure / edge case	Correction
mask_imei	None raises AttributeError; arbitrary characters leak through.	Guard types; accept cleaned ASCII digits; return empty string otherwise.
parse_parts_order	Empty/missing/extra separators, None, bytes and invalid quantities raise exceptions.	Validate each entry; skip malformed items; safely convert positive quantities.
parse_model	Empty or single-word input raises ValueError; None/bytes crash; tabs split incorrectly.	Guard types; split on whitespace; return a consistent two-string tuple.

Results: Original module: all three fuzz tests fail. Fixed module: 23 tests pass, comprising 3 properties with 500 passing generated examples each and 20 regression cases. Baseline commit: ffb09ca. Full minimized failures and final statistics are preserved in evidence/.

Learning reflections and input contracts

Reflection 1: What bugs did fuzz testing reveal?
The original functions trusted input types and formats. Hypothesis automatically reduced failures to None, empty strings and empty bytes. These expose method calls on unsupported types and tuple unpacking with too few fields. It also found a non-digit string accepted by the masking function. Deterministic regressions additionally check invalid numeric quantities, names containing x, duplicate parts, Unicode and whitespace.

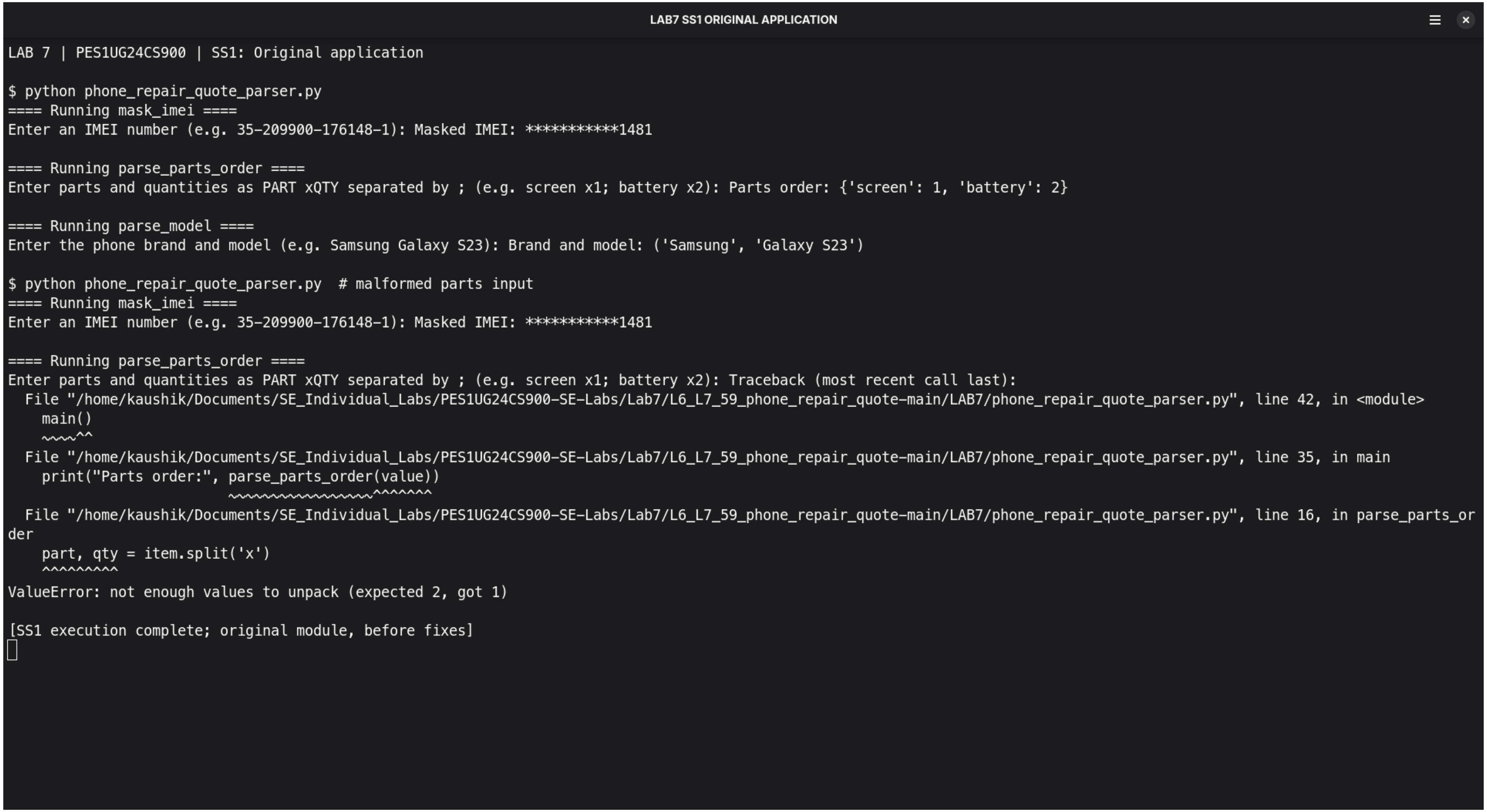
Reflection 2: How does fuzz testing complement ordinary unit tests?
A property describes a general requirement, such as never crashing and returning a consistent output type. Hypothesis generates varied inputs and shrinks failures into small counterexamples. The three properties cover arbitrary Unicode, None, numbers, bytes and containers; the parts strategy also generates valid orders. Twenty fixed cases check concrete results. Passing finite samples does not prove correctness for all inputs.

Reflection 3: How were the fixes validated and what are their limits?
The same tests were run before and after the fixes. Type checks prevent unsupported method calls; structural checks prevent unsafe unpacking; quantity validation and a narrow ValueError handler protect conversion. The corrected CLI repeats the malformed input that crashed originally and reaches the model prompt. All final tests pass. These are parsing helpers: IMEI checksum validation and a parts catalogue are outside scope.

Contracts: IMEI input loses ASCII spaces/dashes, then must contain only ASCII digits; invalid input returns an empty string. Up to four digits remain visible. Parts entries require whitespace before lowercase x and a positive ASCII integer of at most 4300 digits; invalid entries are skipped, and the last valid duplicate wins. Model whitespace is normalized; a brand alone has an empty model, and absent/non-string input returns two empty strings.

Prompt note: The README requests three reflection answers, but the supplied PDF and instruction image list no reflection questions. The three prompts above are supplementary reflections on the stated learning outcomes, not quotations from the handout.

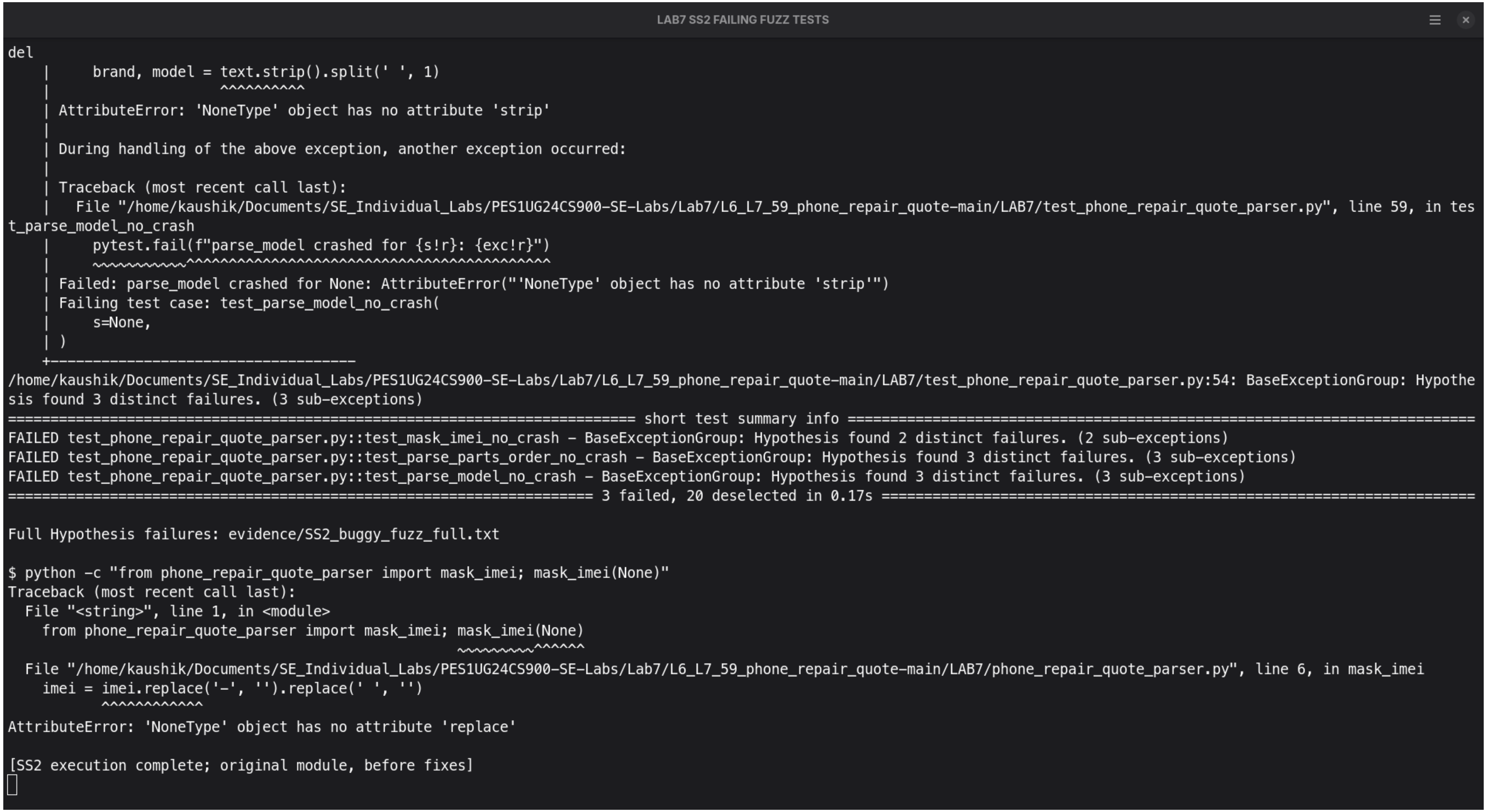
SS1: Original application



```
LAB 7 | PES1UG24CS900 | SS1: Original application

$ python phone_repair_quote_parser.py
==== Running mask_imei ====
Enter an IMEI number (e.g. 35-209900-176148-1): Masked IMEI: ***********1481

==== Running parse_parts_order ====
Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): Parts order: {'screen': 1, 'battery': 2}

==== Running parse_model ====
Enter the phone brand and model (e.g. Samsung Galaxy S23): Brand and model: ('Samsung', 'Galaxy S23')

$ python phone_repair_quote_parser.py  # malformed parts input
==== Running mask_imei ====
Enter an IMEI number (e.g. 35-209900-176148-1): Masked IMEI: ***********1481

==== Running parse_parts_order ====
Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): Traceback (most recent call last):
  File "/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/phone_repair_quote_parser.py", line 42, in <module>
    main()
    ~~~~^^
  File "/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/phone_repair_quote_parser.py", line 35, in main
    print("Parts order:", parse_parts_order(value))
                          ~~~~~~~~~~~~~~~~~^^^^^^^
  File "/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/phone_repair_quote_parser.py", line 16, in parse_parts_or
der
    part, qty = item.split('x')
    ^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)

[SS1 execution complete; original module, before fixes]
```

Valid inputs run successfully. Repeating with parts input 'screen' raises ValueError before the model prompt.

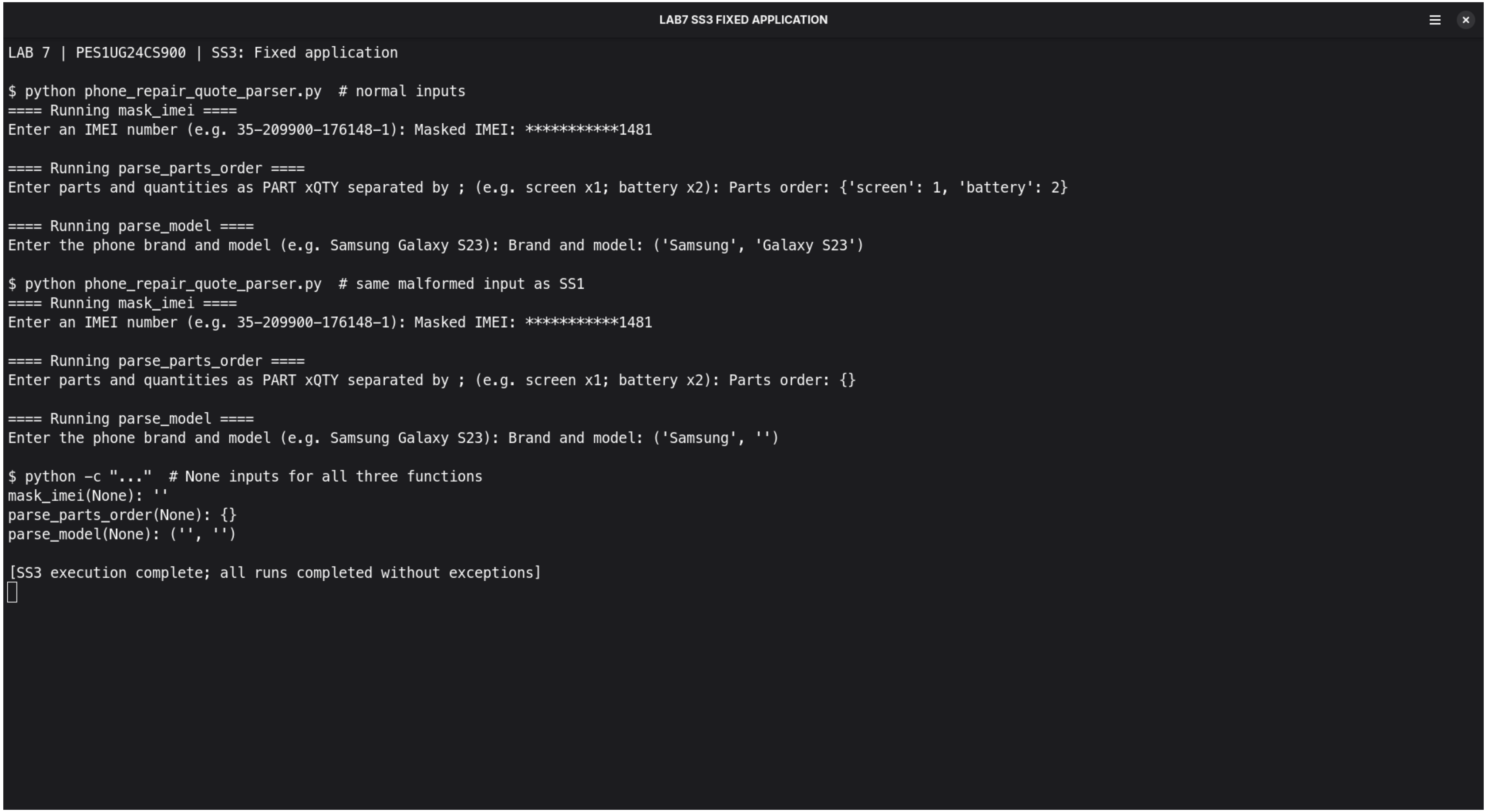
SS2: Failing Hypothesis tests



```
del
    |     brand, model = text.strip().split(' ', 1)
    |                    ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/test_phone_repair_quote_parser.py", line 59, in tes
t_parse_model_no_crash
    |     pytest.fail(f"parse_model crashed for {s!r}: {exc!r}")
    |     ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    | Failed: parse_model crashed for None: AttributeError("'NoneType' object has no attribute 'strip'")
    | Failing test case: test_parse_model_no_crash(
    |     s=None,
    | )
    +------------------------------------
/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/test_phone_repair_quote_parser.py:54: BaseExceptionGroup: Hypothe
sis found 3 distinct failures. (3 sub-exceptions)
========================================================================== short test summary info ===========================================================================
FAILED test_phone_repair_quote_parser.py::test_mask_imei_no_crash - BaseExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_phone_repair_quote_parser.py::test_parse_parts_order_no_crash - BaseExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
FAILED test_phone_repair_quote_parser.py::test_parse_model_no_crash - BaseExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
===================================================================== 3 failed, 20 deselected in 0.17s =======================================================================

Full Hypothesis failures: evidence/SS2_buggy_fuzz_full.txt

$ python -c "from phone_repair_quote_parser import mask_imei; mask_imei(None)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from phone_repair_quote_parser import mask_imei; mask_imei(None)
                                                     ~~~~~~~~~^^^^^^
  File "/home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7/phone_repair_quote_parser.py", line 6, in mask_imei
    imei = imei.replace('-', '').replace(' ', '')
           ^^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'replace'

[SS2 execution complete; original module, before fixes]
```

The three fuzz tests fail on the original module. The terminal shows the minimized None model failure, the three-test summary, and a direct IMEI failure. Full details: SS2_buggy_fuzz_full.txt.

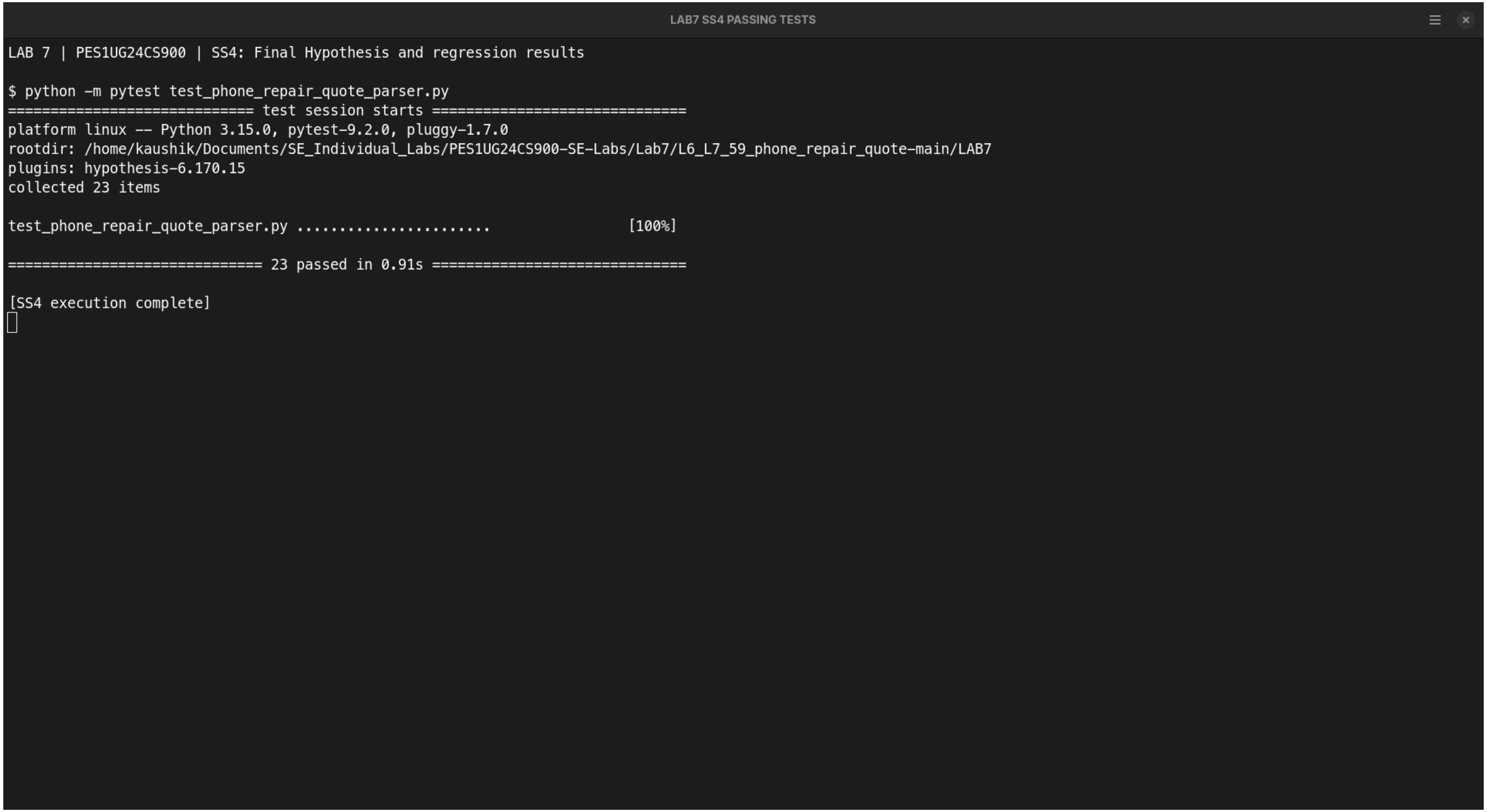
SS3: Corrected application



```
LAB 7 | PES1UG24CS900 | SS3: Fixed application

$ python phone_repair_quote_parser.py  # normal inputs
==== Running mask_imei ====
Enter an IMEI number (e.g. 35-209900-176148-1): Masked IMEI: ***********1481

==== Running parse_parts_order ====
Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): Parts order: {'screen': 1, 'battery': 2}

==== Running parse_model ====
Enter the phone brand and model (e.g. Samsung Galaxy S23): Brand and model: ('Samsung', 'Galaxy S23')

$ python phone_repair_quote_parser.py  # same malformed input as SS1
==== Running mask_imei ====
Enter an IMEI number (e.g. 35-209900-176148-1): Masked IMEI: ***********1481

==== Running parse_parts_order ====
Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): Parts order: {}

==== Running parse_model ====
Enter the phone brand and model (e.g. Samsung Galaxy S23): Brand and model: ('Samsung', '')

$ python -c "..."  # None inputs for all three functions
mask_imei(None): ''
parse_parts_order(None): {}
parse_model(None): ('', '')

[SS3 execution complete; all runs completed without exceptions]
```

Normal inputs retain their results. The same malformed parts input returns an empty order; the single-word model and all three None calls complete safely.

SS4: Final passing test suite



```
LAB 7 | PES1UG24CS900 | SS4: Final Hypothesis and regression results

$ python -m pytest test_phone_repair_quote_parser.py
============================= test session starts ==============================
platform linux -- Python 3.15.0, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/kaushik/Documents/SE_Individual_Labs/PES1UG24CS900-SE-Labs/Lab7/L6_L7_59_phone_repair_quote-main/LAB7
plugins: hypothesis-6.170.15
collected 23 items

test_phone_repair_quote_parser.py .......................                [100%]

============================== 23 passed in 0.91s ==============================

[SS4 execution complete]
```

The full test file passes: three Hypothesis properties and twenty regression cases. Separate final_fuzz_statistics.txt records 500 passing examples for each property.

Source: phone_repair_quote_parser.py

Complete final source | lines 1-46

```python
"""Defensive input parsers for phone repair quotes."""


def mask_imei(imei):
    """
    Removes dashes and spaces, then hides all but the last four digits.
    Non-strings, empty values and non-ASCII digits return an empty string.
    Short digit strings remain visible; length/checksum validation is separate.
    """
    if not isinstance(imei, str):
        return ""
    imei = imei.replace('-', '').replace(' ', '')
    if not imei or any(char not in "0123456789" for char in imei):
        return ""
    return '*' * (len(imei) - 4) + imei[-4:]


def parse_parts_order(text):
    """
    Parses 'screen x1; battery x2' into a {part: quantity} dictionary.
    Skip malformed entries; accept positive ASCII integer quantities only.
    Require whitespace before the quantity marker, allowing names like 'box'.
    For duplicate part names, the last valid quantity wins (original behavior).
    Quantities exceeding 4300 digits are rejected before integer conversion.
    """
    order = {}
    if not isinstance(text, str):
        return order
    for item in text.split(';'):
        part, marker, qty = item.rpartition("x")
        if not marker or not part or not part[-1].isspace():
            continue
        part = part.strip()
        qty = qty.strip()
        if (not part or not qty or len(qty) > 4300
                or any(char not in "0123456789" for char in qty)):
            continue
        try:
            quantity = int(qty)
        except ValueError:
            # Respect a stricter interpreter integer-string conversion limit.
            continue
        if quantity > 0:
            order[part] = quantity
    return order
```

Source: phone_repair_quote_parser.py

Complete final source | lines 47-77

```
def parse_model(text):
    """
    Splits 'Brand Model name' into a (brand, model) tuple.
    Normalize whitespace; a brand alone returns (brand, '').
    Empty and non-string inputs return ('', '').
    """
    if not isinstance(text, str):
        return "", ""
    words = text.split()
    if not words:
        return "", ""
    return words[0].title(), " ".join(words[1:])


def main():
    print("==== Running mask_imei ====")
    value = input("Enter an IMEI number (e.g. 35-209900-176148-1): ")
    print("Masked IMEI:", mask_imei(value))

    print("\n==== Running parse_parts_order ====")
    value = input("Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): ")
    print("Parts order:", parse_parts_order(value))

    print("\n==== Running parse_model ====")
    value = input("Enter the phone brand and model (e.g. Samsung Galaxy S23): ")
    print("Brand and model:", parse_model(value))


if __name__ == "__main__":
    main()
```

Source: test_phone_repair_quote_parser.py

Complete final source | lines 1-46

```
import pytest
from hypothesis import given, settings, strategies as st
from phone_repair_quote_parser import mask_imei, parse_parts_order, parse_model


# Include the handout's arbitrary Unicode/None inputs and other common types.
inputs = st.one_of(
    st.text(), st.none(), st.integers(), st.floats(), st.binary(),
    st.lists(st.integers()), st.dictionaries(st.text(), st.integers()),
)


@settings(max_examples=500)
@given(inputs)
def test_mask_imei_no_crash(s):
    try:
        result = mask_imei(s)
    except Exception as exc:
        pytest.fail(f"mask_imei crashed for {s!r}: {exc!r}")
    assert isinstance(result, str)
    if not isinstance(s, str):
        assert result == ""
    else:
        cleaned = s.replace("-", "").replace(" ", "")
        if not cleaned or any(c not in "0123456789" for c in cleaned):
            assert result == ""
        else:
            visible = min(4, len(cleaned))
            assert result.endswith(cleaned[-visible:])
            assert len(result) == len(cleaned)
            assert result[:-visible] == "*" * (len(cleaned) - visible)


@settings(max_examples=500)
@given(st.one_of(inputs, st.builds(
    lambda part, qty: f"{part} x{qty}",
    st.sampled_from(["screen", "battery", "flex cable", "box"]),
    st.integers(min_value=1, max_value=10000),
)))
def test_parse_parts_order_no_crash(s):
    try:
        result = parse_parts_order(s)
    except Exception as exc:
        pytest.fail(f"parse_parts_order crashed for {s!r}: {exc!r}")
    assert isinstance(result, dict)
    assert all(isinstance(part, str) and part and part == part.strip()
```

Source: test_phone_repair_quote_parser.py

Complete final source | lines 47-90

```
               and isinstance(qty, int) and qty > 0
               for part, qty in result.items())
    if not isinstance(s, str):
        assert result == {}


@settings(max_examples=500)
@given(inputs)
def test_parse_model_no_crash(s):
    try:
        result = parse_model(s)
    except Exception as exc:
        pytest.fail(f"parse_model crashed for {s!r}: {exc!r}")
    assert isinstance(result, tuple) and len(result) == 2
    assert all(isinstance(value, str) for value in result)
    words = s.split() if isinstance(s, str) else []
    expected = (words[0].title(), " ".join(words[1:])) if words else ("", "")
    assert result == expected


@pytest.mark.parametrize("function,value,expected", [
    (mask_imei, "35-209900-176148-1", "***********1481"),
    (mask_imei, "1234", "1234"),
    (mask_imei, "12", "12"),
    (mask_imei, " - ", ""),
    (mask_imei, "abc1234", ""),
    (mask_imei, "\uff11\uff12\uff13\uff14", ""),
    (mask_imei, 123456, ""),
    (parse_parts_order, "screen x1; battery x2", {"screen": 1, "battery": 2}),
    (parse_parts_order, "box x2; flex cable x3", {"box": 2, "flex cable": 3}),
    (parse_parts_order, "screen x1; broken; battery x2;", {"screen": 1, "battery": 2}),
    (parse_parts_order, "screen x1; screen x3", {"screen": 3}),
    (parse_parts_order, "x2; camera x0; screen x-1; battery xno", {}),
    (parse_parts_order, "screen x1.5; battery x²; camera x1x2", {}),
    (parse_parts_order, "screen x" + "9" * 5000, {}),
    (parse_parts_order, None, {}),
    (parse_model, "Samsung Galaxy S23", ("Samsung", "Galaxy S23")),
    (parse_model, "  samsung\tGalaxy   S23  ", ("Samsung", "Galaxy S23")),
    (parse_model, "Samsung", ("Samsung", "")),
    (parse_model, "  ", ("", "")),
    (parse_model, None, ("", "")),
])
def test_regressions(function, value, expected):
    assert function(value) == expected
```